LAPORAN PRAKTIKUM DATA PROCESSING WORKFLOW

MINI PROJECT

Dosen Pengampu : Pak Isbat Uzzin Nadhori S.Kom., M.T



Oleh:

Syavikarani Cahya Putri

NRP 3324600054

Sains Data Terapan B

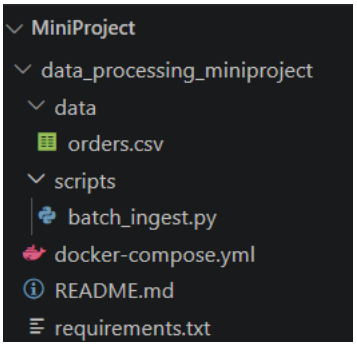
PROGRAM STUDI D4 SAINS DATA TERAPAN

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

1. Persiapan tools dan project
   Kita menggunakan file dataset dan konfigurasi Docker yang sudah disediakan di Paket_Pendukung_Mini_Project.zip. Jadi, kamu tidak perlu membuat dataset dari nol. Dataset ini berisi transaksi valid dan beberapa transaksi yang sengaja dibuat tidak valid untuk pengujian.

   a. Ekstrak file ZIP



   b. Pastikan aplikasi sudah terpasang

```
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject> cd data_processing_miniproject
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python --version
Python 3.10.0
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker --version
Docker version 29.5.2, build 79eb04c
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker compose version
Docker Compose version v5.1.3
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

   c. Buat virtual environment

```
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python -m venv .venv
PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

   d. Instal library

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\vika\onedrive\文件\semester 5\data processing workflow\miniproject\data_processing_miniproject\.venv\lib\site-packages (21.2.3)
Collecting pip
  Downloading pip-26.2.2-py3-none-any.whl (1.8 MB)
     |████████████████████████████████| 1.8 MB 1.1 MB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 21.2.3
    Uninstalling pip-21.2.3:
      Successfully uninstalled pip-21.2.3
Successfully installed pip-26.2.2
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> pip install -r requirements.txt
Collecting pandas>=2.2 (from -r requirements.txt (line 1))
  Using cached pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
Collecting pyarrow>=16 (from -r requirements.txt (line 2))
  Downloading pyarrow-25.0.1-cp310-cp310-win_amd64.whl.metadata (3.0 kB)
Collecting pydantic>=2.7 (from -r requirements.txt (line 3))
  Downloading pydantic-2.14.1-py3-none-any.whl.metadata (133 kB)
Collecting polars>=1.0 (from -r requirements.txt (line 4))
  Downloading polars-2.0.0-py3-none-any.whl.metadata (11 kB)
```

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> pip install -r requirements.txt
8182e19b6643acd21cab9d8267aa14b4a643e8a81f2ac184cf662107748e8c1
  Stored in directory: c:\users\vika\appdata\local\pip\cache\wheels\13\81\2d\b1080c0198f454dbef932811a111a639beedefdb71bfecea3f
Successfully built dbt-core-experimental-parser
Installing collected packages: text-unidecode, pytz, pytimeparse, parsedatetime, leather, daff, zipp, urllib3, tzdata, typing-extensions, tabulate, sqlparse, sqlglot, six, rpds-py, rapidfuzz, pyyaml, python-slugify, python-dotenv, pyarrow, protobuf, polars-runtime-32, pathspec, packaging, orderly-set, numpy, networkx, msgpack, more-itertools, MarkupSafe, kafka-python, isodate, idna, duckdb, dbt-extractor, dbt-core-experimental-parser, colorama, click, charset_normalizer, certifi, Babel, attrs, annotated-types, typing-inspection, requests, referencing, python-dateutil, pydantic-core, polars, opentelemetry-api, mashumaro, jinja2, importlib-metadata, deepdiff, dbt-protos, agate, snowplow-tracker, pydantic, pandas, jsonschema-specifications, jsonschema, metricflow, dbt-common, dbt-adapters, dbt-core, dbt-duckdb
Successfully installed Babel-2.18.0 MarkupSafe-3.0.4 agate-1.9.1 annotated-types-0.8.0 attrs-26.1.0 certifi-2026.10.23 charset_normalizer-3.5.2 click-8.5.0 colorama-0.4.6 daff-1.4.2 dbt-adapters-1.24.5 dbt-common-1.39.0 dbt-core-1.12.6 dbt-core-experimental-parser-2.0.8 dbt-duckdb-1.11.0 dbt-extractor-0.6.0 dbt-protos-1.0.615 deepdiff-8.6.2 duckdb-1.5.6 idna-3.20 importlib-metadata-9.0.1 isodate-0.7.2 jinja2-3.1.6 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 kafka-python-3.0.11 leather-0.4.1 mashumaro-3.17 metricflow-0.213.0 more-itertools-10.8.0 msgpack-1.2.3 networkx-3.4.2 numpy-2.2.6 opentelemetry-api-1.45.1 orderly-set-5.5.0 packaging-26.3 pandas-2.3.3 parsedatetime-2.6 pathspec-1.0.4 polars-2.0.0 polars-runtime-32-2.0.0 protobuf-6.33.6 pyarrow-25.0.1 pydantic-2.14.1 pydantic-core-2.50.1 python-dateutil-2.9.0.post0 python-dotenv-1.2.4 python-slugify-9.1.3 pytimeparse-1.1.8 pytz-2026.5 pyyaml-6.0.3 rapidfuzz-3.14.5 referencing-0.37.0 requests-2.34.2 rpds-py-0.30.0 six-1.17.0 snowplow-tracker-1.1.0 sqlglot-30.24.0 sqlparse-0.6.0 tabulate-0.10.0 text-unidecode-1.3 typing-extensions-4.16.0 typing-inspection-0.4.4 tzdata-2026.5 urllib3-2.8.0 zipp-4.1.2
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python -c "import pandas, pyarrow, pydantic, polars, duckdb; print('Library utama berhasil diimpor')"
Library utama berhasil diimpor
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

2. Memahami dataset

orders.csv

data_processing_miniproject > data > orders.csv > data

```
order_id,customer_id,order_ts,product_category,quantity,unit_price,discount_pct,payment_method,status
O1001,C001,2026-09-01T09:15:00Z,Electronics,1,250.0,0.05,card,paid
O1002,C002,2026-09-01T10:10:00Z,Home,2,45.5,0.0,transfer,shipped
O1003,C003,2026-09-01T11:05:00Z,Books,3,12.99,0.1,ewallet,paid
O1004,C001,2026-09-02T08:45:00Z,Electronics,1,89.9,0.15,card,paid
O1005,C004,2026-09-02T12:00:00Z,Fashion,2,39.0,0.2,ewallet,cancelled
O1006,C005,2026-09-02T13:30:00Z,Home,1,120.0,0.0,transfer,shipped
O1007,C006,2026-09-03T07:20:00Z,Books,5,8.5,0.0,card,paid
O1008,C002,2026-09-03T09:40:00Z,Fashion,1,75.0,0.1,ewallet,paid
O1009,C007,2026-09-03T15:10:00Z,Electronics,1,499.0,0.05,card,refunded
O1010,C008,2026-09-04T10:25:00Z,Home,4,15.75,0.0,transfer,paid
O1011,C009,2026-09-04T12:40:00Z,Books,2,18.0,0.05,card,shipped
O1012,C010,2026-09-04T16:00:00Z,Fashion,3,22.0,0.1,ewallet,paid
O1013,C003,2026-09-05T08:15:00Z,Electronics,1,129.0,0.0,card,paid
O1014,C011,2026-09-05T09:55:00Z,Home,2,64.0,0.05,transfer,shipped
O1015,C012,2026-09-05T14:20:00Z,Books,1,25.0,0.0,card,paid
```

a. Buat file untuk memeriksa dataset. File baru bernama scripts/check_data.py.

batch_ingest.py | check_data.py

data_processing_miniproject > scripts > check_data.py > ...

```python
from pathlib import Path
import pandas as pd

DATA_PATH = Path("data/orders.csv")

df = pd.read_csv(DATA_PATH)

print("=== PEMERIKSAAN DATASET ===")
print("Lokasi file :", DATA_PATH)
print("Jumlah baris:", len(df))
print("Jumlah kolom:", len(df.columns))

print("\nNama kolom:")
print(df.columns.tolist())

print("\nLima data pertama:")
print(df.head())

print("\nJumlah nilai kosong per kolom:")
print(df.isnull().sum())

print("\nDistribusi status pesanan:")
print(df["status"].value_counts(dropna=False))
```

b. Jalankan dari folder utama project:

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/check_data.py
=== PEMERIKSAAN DATASET ===
Lokasi file : data\orders.csv
Jumlah baris: 36
Jumlah kolom: 9

Nama kolom:
['order_id', 'customer_id', 'order_ts', 'product_category', 'quantity', 'unit_price', 'discount_pct', 'payment_method', 'status']

Lima data pertama:
  order_id customer_id              order_ts product_category  quantity  unit_price  discount_pct payment_method     status
0    O1001        C001  2026-09-01T09:15:00Z      Electronics         1      250.00          0.05           card       paid
1    O1002        C002  2026-09-01T10:10:00Z             Home         2       45.50          0.00       transfer    shipped
2    O1003        C003  2026-09-01T11:05:00Z            Books         3       12.99          0.10        ewallet       paid
3    O1004        C001  2026-09-02T08:45:00Z      Electronics         1       89.90          0.15           card       paid
4    O1005        C004  2026-09-02T12:00:00Z          Fashion         2       39.00          0.20        ewallet  cancelled

Jumlah nilai kosong per kolom:
order_id            0
customer_id         1
order_ts            0
product_category    0
quantity            0
unit_price          0
discount_pct        0
payment_method      0
status              0
dtype: int64

Distribusi status pesanan:
status
paid         25
shipped       8
cancelled     1
refunded      1
pending       1
Name: count, dtype: int64
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

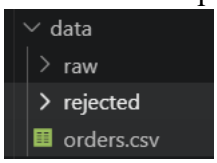
Pemeriksaan Dataset

Pada tahap awal, dilakukan pemeriksaan terhadap dataset transaksi e-commerce yang digunakan sebagai sumber data. Pemeriksaan meliputi jumlah record, struktur kolom, nilai kosong, dan distribusi status pesanan. Dataset sengaja memuat beberapa record yang tidak memenuhi aturan data agar proses validasi dapat diuji. Data dipertahankan dalam kondisi awal sehingga proses ingestion dan data contract dapat diuji secara menyeluruh. Hasil yang diharapkan adalah program menampilkan jumlah data, nama kolom, lima baris pertama, dan ringkasan nilai kosong.

3. Batch ingestion dan penyimpanan Parquet

   Tahap pertama dalam pipeline adalah mengambil data dari sumber secara bertahap, seperti proses pagination pada REST API. Karena proyek ini menggunakan CSV lokal, kita mensimulasikan API dengan pembacaan per halaman. Ini sesuai dengan brief praktikum, tetapi perlu dijelaskan dalam laporan bahwa belum ada permintaan HTTP ke API eksternal.

   a. Buat folder output

   Folder raw akan digunakan untuk menyimpan data hasil ekstraksi dalam format Parquet.

   b. Lengkapi file scripts/batch_ingest.py

   Ganti isi file scripts/batch_ingest.py dengan kode berikut. Kode ini melakukan pagination, retry sederhana, deduplikasi berdasarkan order_id, dan menyimpan hasil ke Parquet.

```python
def get_page(df, page, page_size, retries=3):
    """Membaca satu halaman data dari CSV lokal."""

    for attempt in range(retries):
        try:
            start = page * page_size
            end = start + page_size

            return df.iloc[start:end].copy()

        except Exception:
            if attempt == retries - 1:
                raise

            wait_time = 2 ** attempt
            print(f"Retry dalam {wait_time} detik...")
            time.sleep(wait_time)


def main():
    parser = argparse.ArgumentParser()

    parser.add_argument(
        "--source",
        default="data/orders.csv"
    )
```

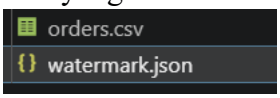
c. Jalankan ingestion

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/batch_ingest.py --source data/orders.csv --out data/raw --page-size 10
Page 1: 10 record
Page 2: 10 record
Page 3: 10 record
Page 4: 6 record

=== HASIL BATCH INGESTION ===
Total sumber : 36
Record baru  : 36
Total unik   : 36
Output       : data\raw\orders.parquet
Watermark    : 2026-09-11T15:00:00+00:00
```

Amati log halaman. Dengan ukuran halaman 10, data akan dibaca dalam beberapa halaman, diikuti satu pembacaan terakhir yang kosong untuk menandai akhir data.

File yang dihasilkan :



d. Periksa hasil Parquet

Buat scripts/check_parquet.py

```python
import pandas as pd

path = "data/raw/orders.parquet"

df = pd.read_parquet(path)

print("=== HASIL PARQUET ===")
print("Jumlah record:", len(df))
print(df.head())
print("\nTipe data:")
print(df.dtypes)
```

Jalankan :

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/check_parquet.py
=== HASIL PARQUET ===
Jumlah record: 36
  order_id customer_id                order_ts product_category  quantity  unit_price  discount_pct payment_method     status
0    O1001        C001 2026-09-01 09:15:00+00:00      Electronics         1      250.00          0.05           card       paid
1    O1002        C002 2026-09-01 10:10:00+00:00            Home         2       45.50          0.00       transfer    shipped
2    O1003        C003 2026-09-01 11:05:00+00:00           Books         3       12.99          0.10        ewallet       paid
3    O1004        C001 2026-09-02 08:45:00+00:00      Electronics         1       89.90          0.15           card       paid
4    O1005        C004 2026-09-02 12:00:00+00:00         Fashion         2       39.00          0.20        ewallet  cancelled

Tipe data:
order_id                        object
customer_id                     object
order_ts           datetime64[ns, UTC]
product_category                object
quantity                         int64
unit_price                     float64
discount_pct                   float64
payment_method                  object
status                          object
dtype: object
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

e. Uji menjalankan ulang

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/batch_ingest.py --source data/orders.csv --out data/raw --page-size 10
Page 1: 10 record
Page 2: 10 record
Page 3: 10 record
Page 4: 6 record

=== HASIL BATCH INGESTION ===
Total sumber : 36
Record baru  : 1
Total unik   : 36
Output       : data\raw\orders.parquet
Watermark    : 2026-09-11T15:00:00+00:00
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Implementasi Batch Ingestion

Batch ingestion dilakukan dengan membaca dataset CSV secara bertahap menggunakan parameter page dan page size. Data hasil ekstraksi disimpan dalam format Parquet untuk mempertahankan struktur tipe data dan mendukung pembacaan kolumnar. Program juga menerapkan checkpoint watermark dan deduplikasi berdasarkan order_id untuk mengurangi pemrosesan berulang. Pengujian dilakukan dengan menjalankan program dua kali dan membandingkan jumlah record unik yang tersimpan.

4. Membuat data contract menggunakan Pydantic

a. Buat file scripts/validate.py

data_processing_miniproject > scripts > validate.py > ...

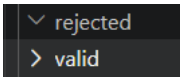
```python
class Order(BaseModel):
    model_config = ConfigDict(extra="forbid")

    order_id: str = Field(min_length=1)
    customer_id: str = Field(min_length=1)
    order_ts: datetime
    product_category: str = Field(min_length=1)
    quantity: int = Field(gt=0)
    unit_price: float = Field(ge=0)
    discount_pct: float = Field(ge=0, le=1)
    payment_method: str = Field(min_length=1)

    status: Literal[
        "paid",
        "shipped",
        "cancelled",
        "refunded"
    ]


def is_blank(value):
    return (
```

b. Buat folder output



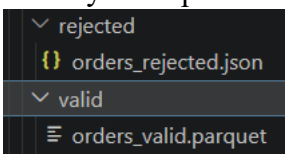
c. Jalankan validasi

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/validate
.py

=== HASIL VALIDASI ===
Total record : 36
Valid        : 30
Rejected     : 6
File valid   : data\valid\orders_valid.parquet
File rejected: data\rejected\orders_rejected.json

Contoh alasan penolakan:
O1031 -> 1 validation error for Order
quantity
  Input should be greater than 0 [type=greater_than, input_value=0, input_type=int]
    For further information visit https://errors.pydantic.dev/2.14/v/greater_than
O1032 -> 1 validation error for Order
unit_price
  Input should be greater than or equal to 0 [type=greater_than_equal, input_value=-12.0, input_type=float]
    For further information visit https://errors.pydantic.dev/2.14/v/greater_than_equal
O1033 -> 1 validation error for Order
discount_pct
  Input should be less than or equal to 1 [type=less_than_equal, input_value=1.2, input_type=float]
    For further information visit https://errors.pydantic.dev/2.14/v/less_than_equal
O1035 -> 1 validation error for Order
status
  Input should be 'paid', 'shipped', 'cancelled' or 'refunded' [type=literal_error, input_value='pending', input_type=str]
    For further information visit https://errors.pydantic.dev/2.14/v/literal_error
O1036 -> Field wajib kosong: customer_id
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Hasilnya berupa dua file terpisah:



5. Menjalankan Kafka dengan Docker

a. Periksa isi docker-compose.yml, kemudian jalankan diterminal

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker compose config
name: data_processing_miniproject
services:
  kafka:
    container_name: dpw-kafka
    environment:
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: "0"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_NODE_ID: "1"
      KAFKA_NUM_PARTITIONS: "3"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
    healthcheck:
      test:
        - CMD-SHELL
        - /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list >/dev/null 2>&1
      timeout: 5s
      interval: 10s
      retries: 10
    image: apache/kafka:3.9.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 9092
        published: "9092"
        protocol: tcp
networks:
  default:
    name: data_processing_miniproject_default
```

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker compose up -d
[+] up 13/13
 ✓ Image apache/kafka:3.9.0                    Pulled                                                    51.0s
 ✓ Network data_processing_miniproject_default Created                                                    0.2s
 ✓ Container dpw-kafka                         Started                                                    6.6s

What's next:
    Filter, search, and stream logs from all your Compose services
    in one place with Docker Desktop's Logs view. docker-desktop:///dashboard/logs
```

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker compose ps
NAME        IMAGE                COMMAND                  SERVICE   CREATED          STATUS                    PORTS
dpw-kafka   apache/kafka:3.9.0   "/__cacert_entrypoin…"   kafka     45 seconds ago   Up 39 seconds (healthy)   0.0.0.0:9092->9092/tcp, [::
:9092->9092/tcp
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

b. Pastikan broker siap

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> docker compose logs --t
ail=50
dpw-kafka  |    transaction.partition.verification.enable = true
dpw-kafka  |    transaction.remove.expired.transaction.cleanup.interval.ms = 3600000
dpw-kafka  |    transaction.state.log.load.buffer.size = 5242880
dpw-kafka  |    transaction.state.log.min.isr = 1
dpw-kafka  |    transaction.state.log.num.partitions = 50
dpw-kafka  |    transaction.state.log.replication.factor = 1
dpw-kafka  |    transaction.state.log.segment.bytes = 104857600
dpw-kafka  |    transactional.id.expiration.ms = 604800000
dpw-kafka  |    unclean.leader.election.enable = false
dpw-kafka  |    unclean.leader.election.interval.ms = 300000
dpw-kafka  |    unstable.api.versions.enable = false
dpw-kafka  |    unstable.feature.versions.enable = false
dpw-kafka  |    zookeeper.clientCnxnSocket = null
dpw-kafka  |    zookeeper.connect = null
dpw-kafka  |    zookeeper.connection.timeout.ms = null
dpw-kafka  |    zookeeper.max.in.flight.requests = 10
dpw-kafka  |    zookeeper.metadata.migration.enable = false
dpw-kafka  |    zookeeper.metadata.migration.min.batch.size = 200
dpw-kafka  |    zookeeper.session.timeout.ms = 18000
dpw-kafka  |    zookeeper.set.acl = false
dpw-kafka  |    zookeeper.ssl.cipher.suites = null
dpw-kafka  |    zookeeper.ssl.client.enable = false
```

6. Membuat Kafka Producer

   Producer adalah program yang mengirimkan transaksi sebagai event ke Kafka.

   a. Buat file scripts/producer.py

```
data_processing_miniproject > scripts > producer.py > ...
 9
10  BOOTSTRAP_SERVERS = "localhost:9092"
11  TOPIC = "orders"
12  SOURCE = Path("data/orders.csv")
13
14
15  def json_serializer(value):
16      return json.dumps(value, default=str).encode("utf-8")
17
18
19  def main():
20      if not SOURCE.exists():
21          raise FileNotFoundError(
22              f"Dataset tidak ditemukan: {SOURCE}"
23          )
24
25      df = pd.read_csv(SOURCE)
26
27      # Ambil minimal 20 record untuk demo.
28      records = df.head(20).to_dict(orient="records")
29
30      producer = KafkaProducer(
31          bootstrap_servers=BOOTSTRAP_SERVERS,
32          value_serializer=json_serializer
```

   b. Jalankan producer

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/producer
.py
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\producer.py:68: Deprecation
Warning: key_serializer does not implement kafka.serializer.Serializer
  main()
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\producer.py:68: Deprecation
Warning: value_serializer does not implement kafka.serializer.Serializer
  main()
Event terkirim | order_id=O1001 | partition=0 | offset=0
Event terkirim | order_id=O1002 | partition=2 | offset=0
Event terkirim | order_id=O1003 | partition=2 | offset=1
Event terkirim | order_id=O1004 | partition=0 | offset=1
Event terkirim | order_id=O1005 | partition=2 | offset=2
Event terkirim | order_id=O1006 | partition=2 | offset=3
Event terkirim | order_id=O1007 | partition=1 | offset=0
Event terkirim | order_id=O1008 | partition=0 | offset=2
Event terkirim | order_id=O1009 | partition=1 | offset=1
Event terkirim | order_id=O1010 | partition=2 | offset=4
Event terkirim | order_id=O1011 | partition=0 | offset=3
Event terkirim | order_id=O1012 | partition=1 | offset=2
Event terkirim | order_id=O1013 | partition=2 | offset=5
Event terkirim | order_id=O1014 | partition=0 | offset=4
Event terkirim | order_id=O1015 | partition=2 | offset=6
Event terkirim | order_id=O1016 | partition=0 | offset=5
Event terkirim | order_id=O1017 | partition=0 | offset=6
Event terkirim | order_id=O1018 | partition=2 | offset=7
Event terkirim | order_id=O1019 | partition=2 | offset=8
Event terkirim | order_id=O1020 | partition=2 | offset=9

Selesai mengirim 20 event.
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Hasil yang diharapkan adalah log pengiriman event beserta order_id, partition, dan offset.

7. Membuat Kafka Consumer

   Consumer membaca pesan dari topic orders, memeriksa kontrak data, lalu menyimpan hasil valid dan rejected ke file berbeda.

   a. Buat file scripts/consumer.py

```
data_processing_miniproject > scripts > consumer.py > ...
13  TOPIC = "orders"
14  GROUP_ID = "orders-validation-group"
15
16  VALID_PATH = Path("data/stream/valid.jsonl")
17  REJECTED_PATH = Path("data/stream/rejected.jsonl")
18
19
20  class Order(BaseModel):
21      model_config = ConfigDict(extra="forbid")
22
23      order_id: str = Field(min_length=1)
24      customer_id: str = Field(min_length=1)
25      order_ts: datetime
26      product_category: str = Field(min_length=1)
27      quantity: int = Field(gt=0)
28      unit_price: float = Field(ge=0)
29      discount_pct: float = Field(ge=0, le=1)
30      payment_method: str = Field(min_length=1)
31
32      status: Literal[
33          "paid",
34          "shipped",
35          "cancelled",
36          "refunded"
37      ]
38
39
40  def is_blank(value):
```

   b. Jalankan consumer terlebih dahulu

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/consumer
.py
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\consumer.py:86: Deprecatior
Warning: value_deserializer does not implement kafka.serializer.Deserializer
  consumer = KafkaConsumer(
VALID: O1007 | partition=1 | offset=0
VALID: O1009 | partition=1 | offset=1
VALID: O1012 | partition=1 | offset=2
VALID: O1002 | partition=2 | offset=0
VALID: O1003 | partition=2 | offset=1
VALID: O1005 | partition=2 | offset=2
VALID: O1006 | partition=2 | offset=3
VALID: O1010 | partition=2 | offset=4
VALID: O1013 | partition=2 | offset=5
VALID: O1015 | partition=2 | offset=6
VALID: O1018 | partition=2 | offset=7
VALID: O1019 | partition=2 | offset=8
VALID: O1020 | partition=2 | offset=9
VALID: O1001 | partition=0 | offset=0
VALID: O1004 | partition=0 | offset=1
VALID: O1008 | partition=0 | offset=2
VALID: O1011 | partition=0 | offset=3
VALID: O1014 | partition=0 | offset=4
VALID: O1016 | partition=0 | offset=5
VALID: O1017 | partition=0 | offset=6
```

```
=== RINGKASAN CONSUMER ===
Valid   : 20
Rejected: 0
File valid   : data\stream\valid.jsonl
File rejected: data\stream\rejected.jsonl
```

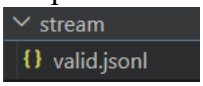
Consumer akan menunggu pesan selama sekitar 15 detik setelah tidak ada pesan yang masuk. Karena auto_offset_reset="earliest", consumer group yang belum memiliki offset akan mulai membaca dari pesan paling awal yang tersedia.

c. Jalankan producer di terminal lain

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/producer.py
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\producer.p
y:68: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  main()
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\producer.p
y:68: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  main()
Event terkirim | order_id=O1001 | partition=0 | offset=7
Event terkirim | order_id=O1002 | partition=2 | offset=10
Event terkirim | order_id=O1003 | partition=2 | offset=11
Event terkirim | order_id=O1004 | partition=0 | offset=8
Event terkirim | order_id=O1005 | partition=2 | offset=12
Event terkirim | order_id=O1006 | partition=2 | offset=13
Event terkirim | order_id=O1007 | partition=1 | offset=3
Event terkirim | order_id=O1008 | partition=0 | offset=9
Event terkirim | order_id=O1009 | partition=1 | offset=4
Event terkirim | order_id=O1010 | partition=2 | offset=14
Event terkirim | order_id=O1011 | partition=0 | offset=10
Event terkirim | order_id=O1012 | partition=1 | offset=5
Event terkirim | order_id=O1013 | partition=2 | offset=15
Event terkirim | order_id=O1014 | partition=0 | offset=11
Event terkirim | order_id=O1015 | partition=2 | offset=16
Event terkirim | order_id=O1016 | partition=0 | offset=12
Event terkirim | order_id=O1017 | partition=0 | offset=13
Event terkirim | order_id=O1018 | partition=2 | offset=17
Event terkirim | order_id=O1019 | partition=2 | offset=18
Event terkirim | order_id=O1020 | partition=2 | offset=19

Selesai mengirim 20 event.
```

Consumer seharusnya mulai menerima event yang dikirim producer.
Output akan tersimpan di:



8. Menguji duplikasi event Kafka

a. Buat file scripts/test_duplicate_event.py

```
data_processing_miniproject > scripts > test_duplicate_event.py > ...
    "order_ts": "2026-10-25T10:00:00",
    "product_category": "Electronics",
    "quantity": 2,
    "unit_price": 100000,
    "discount_pct": 0.1,
    "payment_method": "transfer",
    "status": "paid",
}

producer = KafkaProducer(
    bootstrap_servers=BOOTSTRAP_SERVERS,
    value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    key_serializer=lambda key: key.encode("utf-8"),
    acks="all",
    retries=3,
)

try:
    for i in range(2):
        result = producer.send(
            TOPIC,
            key=event["order_id"],
```

b. Jalankan pengujian

Terminal 1:

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/consumer.py
C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject\scripts\consumer.
y:86: DeprecationWarning: value_deserializer does not implement kafka.serializer.Deserializer
  consumer = KafkaConsumer(
VALID: TEST-DUPLICATE-001 | partition=2 | offset=20
VALID: TEST-DUPLICATE-001 | partition=2 | offset=21

=== RINGKASAN CONSUMER ===
Valid   : 2
Rejected: 0
File valid   : data\stream\valid.jsonl
File rejected: data\stream\rejected.jsonl
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Terminal 2:

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/test_duplicate_event.py
Event ke-1 berhasil dikirim
Order ID : TEST-DUPLICATE-001
Partition: 2
Offset   : 20
------------------------------
Event ke-2 berhasil dikirim
Order ID : TEST-DUPLICATE-001
Partition: 2
Offset   : 21
------------------------------
Pengujian duplikasi selesai.
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Pengujian dilakukan dengan mengirim dua event Kafka yang memiliki order_id sama. Kedua pesan dapat memiliki offset berbeda karena Kafka mencatatnya sebagai pesan terpisah. Consumer yang hanya melakukan validasi data belum otomatis mencegah duplikasi transaksi. Oleh karena itu, diperlukan mekanisme idempotensi pada sink, misalnya dengan menjadikan order_id sebagai kunci unik, agar transaksi yang sama tidak dihitung berulang kali.

9. Transformasi data
   a. Install library

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> pip install polars pyarrow pandas
Requirement already satisfied: polars in .\.venv\lib\site-packages (2.0.0)
Requirement already satisfied: pyarrow in .\.venv\lib\site-packages (25.0.1)
Requirement already satisfied: pandas in .\.venv\lib\site-packages (2.3.3)
Requirement already satisfied: polars-runtime-32==2.0.0 in .\.venv\lib\site-packages (from polars) (2.0.0)
Requirement already satisfied: numpy>=1.22.4 in .\.venv\lib\site-packages (from pandas) (2.2.6)
Requirement already satisfied: python-dateutil>=2.8.2 in .\.venv\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in .\.venv\lib\site-packages (from pandas) (2026.5)
Requirement already satisfied: tzdata>=2022.7 in .\.venv\lib\site-packages (from pandas) (2026.5)
Requirement already satisfied: six>=1.5 in .\.venv\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

b. Buat file scripts/transform_data.py

```python
INPUT_PATH = Path("data/valid/orders_valid.parquet")
OUTPUT_PATH = Path("data/warehouse/orders_transformed.parquet")


def main():
    if not INPUT_PATH.exists():
        raise FileNotFoundError(
            f"File tidak ditemukan: {INPUT_PATH}. "
            "Jalankan validasi data terlebih dahulu."
        )

    df = pd.read_parquet(INPUT_PATH)

    # Normalisasi tipe data
    df["order_ts"] = pd.to_datetime(
        df["order_ts"], utc=True, errors="raise"
    )
    df["quantity"] = pd.to_numeric(df["quantity"]).astype("int64")
```

c. Jalankan:

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python scripts/transform_data.py
Transformasi selesai.
Jumlah data: 30
File output: data\warehouse\orders_transformed.parquet
  order_id  gross_amount  net_amount
0    O1001        250.00     237.500
1    O1002         91.00      91.000
2    O1003         38.97      35.073
3    O1004         89.90      76.415
4    O1005         78.00      62.400
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

10. Membandingkan Pandas dan Polars

a. Buat scripts/benchmark.py

```python
def run_pandas():
    )
    df["gross_amount"] = df["quantity"] * df["unit_price"]
    df["net_amount"] = (
        df["gross_amount"] * (1 - df["discount_pct"])
    )

    # Agregasi agar kedua library melakukan pekerjaan serupa
    result = df.groupby("product_category")["net_amount"].sum()
    return len(df), result


def run_polars():
    result = (
        pl.scan_parquet(INPUT_PATH)
        .with_columns(
            pl.col("order_ts").str.to_datetime(
                time_zone="UTC", strict=True
            ),
            (
                pl.col("quantity") * pl.col("unit_price")
            ).alias("gross_amount"),
```

b. Jalankan dan lihat hasilnya

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/benchmark.py

PANDAS
Jumlah baris: 30
Waktu: 0.266188 detik
Contoh hasil:
product_category
Books          236.733
Electronics   2079.165
Fashion        635.600
Home           996.975
Name: net_amount, dtype: float64

POLARS LAZY
Jumlah baris: 30
Waktu: 0.137906 detik
Contoh hasil:
shape: (4, 2)
┌──────────────────┬────────────┐
│ product_category ┆ net_amount │
│ ---              ┆ ---        │
│ str              ┆ f64        │
╞══════════════════╪════════════╡
│ Home             ┆ 996.975    │
│ Books            ┆ 236.733    │
│ Electronics      ┆ 2079.165   │
│ Fashion          ┆ 635.6      │
└──────────────────┴────────────┘

=== RINGKASAN BENCHMARK ===
Pandas       : 0.266188 detik
Polars Lazy  : 0.137906 detik
Gunakan hasil pengukuran aktual untuk laporan.
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Pengujian membandingkan waktu transformasi dan agregasi menggunakan Pandas dan Polars pada dataset yang sama. Polars menggunakan lazy execution sehingga rencana pemrosesan dapat dioptimalkan sebelum dijalankan. Hasil benchmark menunjukkan performa pada lingkungan pengujian ini; hasil dapat berbeda tergantung ukuran data, perangkat, dan jenis operasi.

11. Eksperimen data skew

    Data skew terjadi ketika distribusi data tidak merata, misalnya satu customer_id memiliki jauh lebih banyak transaksi daripada pelanggan lainnya.

    a. Buat scripts/data_skew.py

```
data_processing_miniproject > scripts > data_skew.py > ...
11  def measure_groupby(df, label):
15          df.groupby("customer_id")
16          .agg(
17              total_orders=("order_id", "count"),
18              total_sales=("net_amount", "sum"),
19          )
20      )
21
22      elapsed = perf_counter() - start
23
24      print(f"\n{label}")
25      print(f"Jumlah baris: {len(df)}")
26      print(f"Jumlah customer: {df['customer_id'].nunique()}")
27      print(f"Waktu groupby: {elapsed:.6f} detik")
28      print(result.sort_values(
29          "total_orders", ascending=False
30      ).head())
31
32      return elapsed
33
34
35  def main():
36      if not INPUT_PATH.exists():
37          raise FileNotFoundError(
38              f"File tidak ditemukan: {INPUT_PATH}"
39          )
40
41      df = pd.read_parquet(INPUT_PATH)
```

b. Jalankan

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>



 …
DATA SKEWED
Jumlah baris: 170
Jumlah customer: 25
Waktu groupby: 0.008362 detik
             total_orders  total_sales
customer_id
C001                   40     4664.150
C002                   10      792.500
C003                   10      820.365
C004                    5      312.000
C005                    5      600.000

=== PERBANDINGAN ===
Normal : 0.035636 detik
Skewed : 0.008362 detik
Dataset skewed disimpan di: data\warehouse\orders_skewed.parquet
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

Eksperimen ini menggandakan baris milik satu pelanggan agar pelanggan tersebut menjadi lebih dominan. Karena data bertambah, waktu agregasi juga bisa berubah; hasilnya perlu diamati, bukan diasumsikan.

12. Membuat Data Warehouse dengan DuckDB

a. Buat scripts/build_warehouse.py

```
data_processing_miniproject > scripts > build_warehouse.py > ...
  9   def main():
 21           # Staging: data valid hasil transformasi
 22           con.execute("""
 23               CREATE OR REPLACE TABLE stg_orders AS
 24               SELECT
 25                   CAST(order_id AS VARCHAR) AS order_id,
 26                   CAST(customer_id AS VARCHAR) AS customer_id,
 27                   CAST(order_ts AS TIMESTAMP WITH TIME ZONE) AS order_ts,
 28                   CAST(product_category AS VARCHAR) AS product_category,
 29                   CAST(quantity AS INTEGER) AS quantity,
 30                   CAST(unit_price AS DOUBLE) AS unit_price,
 31                   CAST(discount_pct AS DOUBLE) AS discount_pct,
 32                   CAST(payment_method AS VARCHAR) AS payment_method,
 33                   CAST(status AS VARCHAR) AS status,
 34                   CAST(gross_amount AS DOUBLE) AS gross_amount,
 35                   CAST(net_amount AS DOUBLE) AS net_amount
 36               FROM read_parquet(?)
 37           """, [INPUT_PATH])
```

b. Jalankan file :

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/build_warehouse.py
stg_orders: 30 baris
dim_customer: 25 baris
dim_product_category: 4 baris
fct_orders: 30 baris
mart_daily_sales: 28 baris

Warehouse tersimpan di: data/warehouse/ecommerce.duckdb

Contoh hasil mart_daily_sales:
  order_date product_category payment_method  total_orders  total_quantity  net_sales
0 2026-09-01            Books        ewallet             1             3.0     35.073
1 2026-09-01      Electronics           card             1             1.0    237.500
2 2026-09-01             Home       transfer             1             2.0     91.000
3 2026-09-02      Electronics           card             1             1.0     76.415
4 2026-09-02             Home       transfer             1             1.0    120.000
5 2026-09-03            Books           card             1             5.0     42.500
6 2026-09-03          Fashion        ewallet             1             1.0     67.500
7 2026-09-04            Books           card             1             2.0     34.200
8 2026-09-04          Fashion        ewallet             1             3.0     59.400
9 2026-09-04             Home       transfer             1             4.0     63.000
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

c. Cek hasil warehouse

Buat scripts/check_warehouse.py:

```
data_processing_miniproject > scripts > check_warehouse.py > ...
1
2   import duckdb
3
4   DB_PATH = "data/warehouse/ecommerce.duckdb"
5
6   con = duckdb.connect(DB_PATH, read_only=True)
7
8   try:
9       print("Jumlah transaksi per status:")
10      print(con.execute("""
11          SELECT status, COUNT(*) AS total
12          FROM fct_orders
13          GROUP BY status
14          ORDER BY total DESC
15      """).df())
16
17      print("\nPenjualan harian:")
18      print(con.execute("""
19          SELECT *
20          FROM mart_daily_sales
21          ORDER BY order_date
22          LIMIT 10
23      """).df())
```

Jalankan:

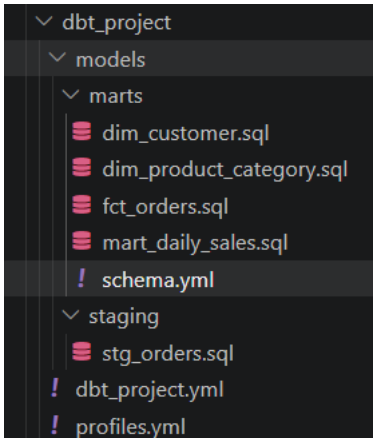
```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> python
 scripts/check_warehouse.py
Jumlah transaksi per status:
      status  total
0       paid     20
1    shipped      8
2  cancelled      1
3   refunded      1

Penjualan harian:
   order_date product_category payment_method  total_orders  total_quantity  net_sales
0  2026-09-01            Books        ewallet             1             3.0     35.073
1  2026-09-01             Home       transfer             1             2.0     91.000
2  2026-09-01      Electronics           card             1             1.0    237.500
3  2026-09-02      Electronics           card             1             1.0     76.415
4  2026-09-02             Home       transfer             1             1.0    120.000
5  2026-09-03          Fashion        ewallet             1             1.0     67.500
6  2026-09-03            Books           card             1             5.0     42.500
7  2026-09-04            Books           card             1             2.0     34.200
8  2026-09-04             Home       transfer             1             4.0     63.000
9  2026-09-04          Fashion        ewallet             1             3.0     59.400
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

DuckDB digunakan sebagai data warehouse lokal untuk menyimpan staging, tabel fakta, tabel dimensi, dan agregasi penjualan harian. Tabel fakta menyimpan transaksi, sedangkan dimensi menyediakan konteks pelanggan dan kategori produk. Agregasi penjualan hanya menghitung pesanan berstatus paid dan shipped, sesuai aturan bisnis yang digunakan pada eksperimen ini.

13. Membuat proyek dbt

Dari folder utama proyek, buat struktur seperti berikut:



a. Buat dbt_project/dbt_project.yml

data_processing_miniproject > dbt_project > ! dbt_project.yml

```yaml
name: ecommerce_warehouse
version: "1.0.0"
config-version: 2

profile: ecommerce_warehouse

model-paths: ["models"]
test-paths: ["tests"]
clean-targets:
  - "target"
  - "dbt_packages"

models:
  ecommerce_warehouse:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

b. Buat dbt_project/profiles.yml

data_processing_miniproject > dbt_project > ! profiles.yml

```yaml
ecommerce_warehouse:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: "data/warehouse/ecommerce_dbt.duckdb"
      schema: main
      threads: 1
```

c. Buat dbt_project/models/staging/stg_orders.sql:

data_processing_miniproject > dbt_project > models > staging > stg_orders.sql

```sql
SELECT
    CAST(order_id AS VARCHAR) AS order_id,
    CAST(customer_id AS VARCHAR) AS customer_id,
    CAST(order_ts AS TIMESTAMP) AS order_ts,
    CAST(product_category AS VARCHAR) AS product_category,
    CAST(quantity AS INTEGER) AS quantity,
    CAST(unit_price AS DOUBLE) AS unit_price,
    CAST(discount_pct AS DOUBLE) AS discount_pct,
    CAST(payment_method AS VARCHAR) AS payment_method,
    CAST(status AS VARCHAR) AS status,
    CAST(quantity * unit_price AS DOUBLE) AS gross_amount,
    CAST(
        quantity * unit_price * (1 - discount_pct)
        AS DOUBLE
    ) AS net_amount
FROM read_parquet('data/warehouse/orders_transformed.parquet')
```

d. Buat file dbt_project/models/marts/dim_customer.sql

data_processing_miniproject > dbt_project > models > marts > dim_customer.sql

```sql

  SELECT DISTINCT
      customer_id
  FROM {{ ref('stg_orders') }}
  WHERE customer_id IS NOT NULL
```

e. Buat dbt_project/models/marts/dim_product_category.sql

data_processing_miniproject > dbt_project > models > marts > dim_product_category.sql

```sql

  SELECT DISTINCT
      product_category
  FROM {{ ref('stg_orders') }}
  WHERE product_category IS NOT NULL
```

f. Buat dbt_project/models/marts/fct_orders.sql

data_processing_miniproject > dbt_project > models > marts > fct_orders.sql

```sql
),

deduplicated AS (
    SELECT *
    FROM ranked_orders
    WHERE row_num = 1
)

SELECT
    o.order_id,
    o.customer_id,
    o.product_category,
    o.order_ts,
    CAST(o.order_ts AS DATE) AS order_date,
    o.quantity,
    o.unit_price,
    o.discount_pct,
    o.gross_amount,
    o.net_amount,
    o.payment_method,
    o.status
FROM deduplicated o
INNER JOIN {{ ref('dim_customer') }} c
    ON o.customer_id = c.customer_id
INNER JOIN {{ ref('dim_product_category') }} p
    ON o.product_category = p.product_category
```

g. Buat dbt_project/models/marts/mart_daily_sales.sql

data_processing_miniproject > dbt_project > models > marts > mart_daily_sales.sql

```sql

SELECT
    order_date,
    product_category,
    payment_method,
    COUNT(*) AS total_orders,
    SUM(quantity) AS total_quantity,
    SUM(net_amount) AS net_sales
FROM {{ ref('fct_orders') }}
WHERE status IN ('paid', 'shipped')
GROUP BY
    order_date,
    product_category,
    payment_method
```

h. Buat dbt_project/models/marts/schema.yml

```
data_processing_miniproject > dbt_project > models > marts > ! schema.yml
  4   models:
 12     - name: dim_product_category
 18
 19   - name: fct_orders
 20     columns:
 21       - name: order_id
 22         data_tests:
 23           - unique
 24           - not_null
 25
 26       - name: customer_id
 27         data_tests:
 28           - not_null
 29           - relationships:
 30               arguments:
 31                 to: ref('dim_customer')
 32                 field: customer_id
 33
 34       - name: product_category
 35         data_tests:
 36           - not_null
 37           - relationships:
 38               arguments:
 39                 to: ref('dim_product_category')
 40                 field: product_category
```

14. Menjalankan dbt
    a. Install dbt-duckdb

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> pip in
stall dbt-duckdb
Requirement already satisfied: annotated-types>=0.6.0 in .\.venv\lib\site-packages (from pydantic<3->dbt-core>=1.8.0->dbt-
duckdb) (0.8.0)
Requirement already satisfied: pydantic-core==2.50.1 in .\.venv\lib\site-packages (from pydantic<3->dbt-core>=1.8.0->dbt-d
uckdb) (2.50.1)
Requirement already satisfied: typing-inspection>=0.4.4 in .\.venv\lib\site-packages (from pydantic<3->dbt-core>=1.8.0->db
t-duckdb) (0.4.4)
Requirement already satisfied: zipp>=3.20 in .\.venv\lib\site-packages (from importlib-metadata>=4.0->metricflow<1.0,>=0.2
13.0->dbt-core>=1.8.0->dbt-duckdb) (4.1.2)
Requirement already satisfied: text-unidecode>=1.3 in .\.venv\lib\site-packages (from python-slugify>=1.2.1->agate<2.0,>=1
.0->dbt-adapters<2,>=1->dbt-duckdb) (1.3)
```

    b. Periksa koneksi dan konfigurasi

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> dbt de
bug --project-dir dbt_project --profiles-dir dbt_project
18:30:58  Using dbt_project.yml file at dbt_project\dbt_project.yml
18:30:58  adapter type: duckdb
18:30:58  adapter version: 1.11.0
18:30:59  Configuration:
18:30:59    profiles.yml file [OK found and valid]
18:30:59    dbt_project.yml file [OK found and valid]
18:30:59  Required dependencies:
18:30:59   - git [OK found]

18:30:59  Connection:
18:30:59    database: ecommerce_dbt
18:30:59    schema: main
18:30:59    path: data/warehouse/ecommerce_dbt.duckdb
18:30:59    config_options: None
18:30:59    extensions: None
18:30:59    settings: {}
18:30:59    external_root: .
18:30:59    use_credential_provider: None
18:30:59    attach: None
18:30:59    filesystems: None
18:30:59    remote: None
18:30:59    plugins: None
18:30:59    disable_transactions: False
18:30:59  Registered adapter: duckdb=1.11.0
18:31:00    Connection test: [OK connection ok]

18:31:00  All checks passed!
```

    c. Jalankan semua model

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> dbt ru
n --project-dir dbt_project --profiles-dir dbt_project
18:31:08  Running with dbt=1.12.6
18:31:09  Registered adapter: duckdb=1.11.0
18:31:10  Unable to do partial parsing because saved manifest not found. Starting full parse.
18:31:15  Found 5 models, 10 data tests, 500 macros
18:31:15
18:31:15  Concurrency: 1 threads (target='dev')
18:31:15
18:31:15  1 of 5 START sql view model main.stg_orders ..................................... [RUN]
18:31:16  1 of 5 OK created sql view model main.stg_orders ................................ [OK in 0.21s]
18:31:16  2 of 5 START sql table model main.dim_customer ................................. [RUN]
18:31:16  2 of 5 OK created sql table model main.dim_customer ............................ [OK in 0.18s]
18:31:16  3 of 5 START sql table model main.dim_product_category ......................... [RUN]
18:31:16  3 of 5 OK created sql table model main.dim_product_category .................... [OK in 0.07s]
18:31:16  4 of 5 START sql table model main.fct_orders ................................... [RUN]
18:31:16  4 of 5 OK created sql table model main.fct_orders .............................. [OK in 0.11s]
18:31:16  5 of 5 START sql table model main.mart_daily_sales ............................. [RUN]
18:31:16  5 of 5 OK created sql table model main.mart_daily_sales ........................ [OK in 0.08s]
18:31:16
18:31:16  Finished running 4 table models, 1 view model in 0 hours 0 minutes and 1.36 seconds (1.36s).
18:31:16
18:31:16  Completed successfully
18:31:16
18:31:16  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=5
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject>
```

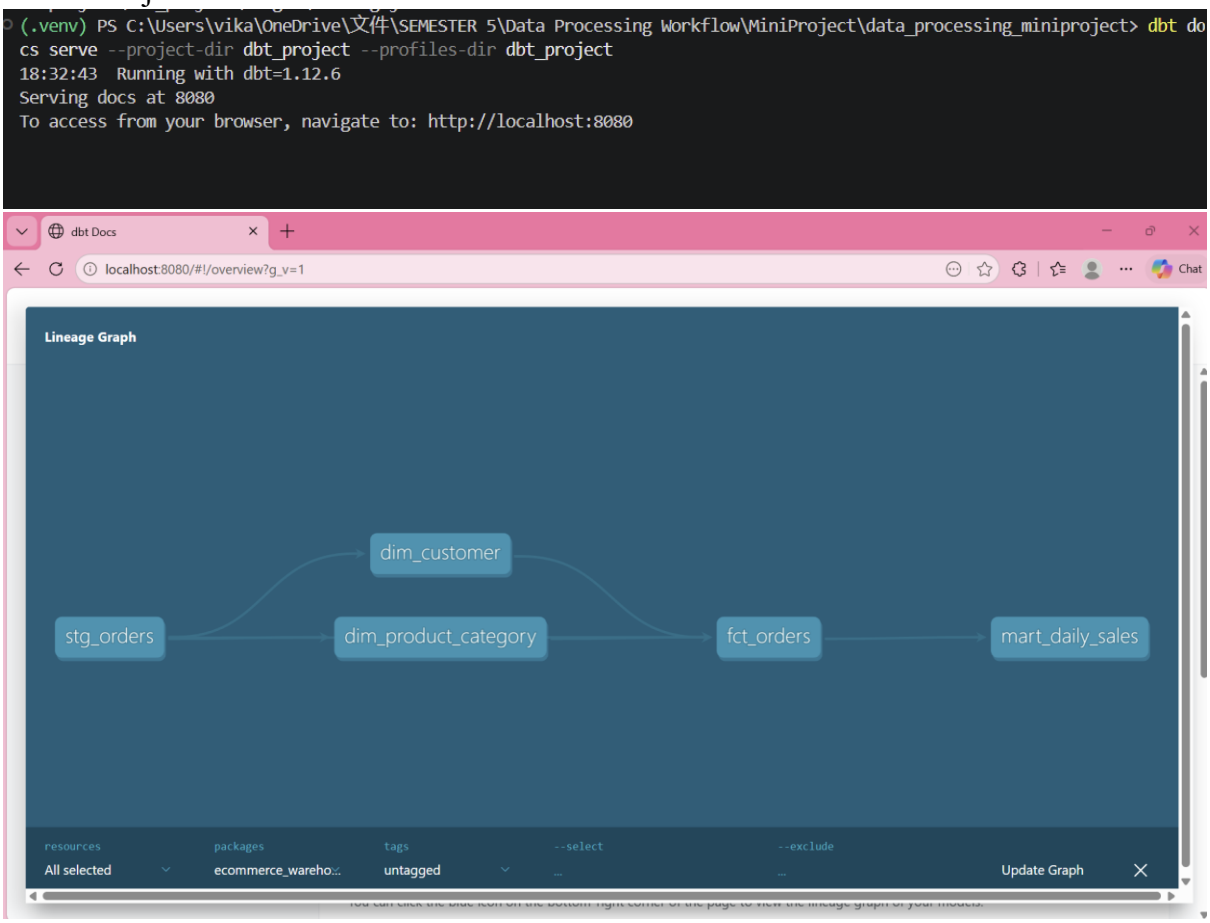
d. Jalankan tes

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> dbt te
st --project-dir dbt_project --profiles-dir dbt_project
18:31:54  Registered adapter: duckdb=1.11.0
18:31:56  Found 5 models, 10 data tests, 500 macros
18:31:56
18:31:56  Concurrency: 1 threads (target='dev')
18:31:56
18:31:57  1 of 10 START test not_null_dim_customer_customer_id ........................... [RUN]
18:31:57  1 of 10 PASS not_null_dim_customer_customer_id ................................. [PASS in 0.10s]
18:31:57  2 of 10 START test not_null_dim_product_category_product_category .............. [RUN]
18:31:57  2 of 10 PASS not_null_dim_product_category_product_category .................... [PASS in 0.03s]
18:31:57  3 of 10 START test not_null_fct_orders_customer_id ............................. [RUN]
18:31:57  3 of 10 PASS not_null_fct_orders_customer_id ................................... [PASS in 0.05s]
18:31:57  4 of 10 START test not_null_fct_orders_order_id ................................ [RUN]
18:31:57  4 of 10 PASS not_null_fct_orders_order_id ..................................... [PASS in 0.03s]
18:31:57  5 of 10 START test not_null_fct_orders_product_category ........................ [RUN]
18:31:57  5 of 10 PASS not_null_fct_orders_product_category ............................. [PASS in 0.04s]
18:31:57  6 of 10 START test relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [RUN]
18:31:57  6 of 10 PASS relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [PASS in 0.06s]
18:31:57  7 of 10 START test relationships_fct_orders_product_category__product_category__ref_dim_product_category_  [RUN]
18:31:57  7 of 10 PASS relationships_fct_orders_product_category__product_category__ref_dim_product_category_  [PASS in 0.
06s]
18:31:57  8 of 10 START test unique_dim_customer_customer_id ............................. [RUN]
18:31:57  8 of 10 PASS unique_dim_customer_customer_id ................................... [PASS in 0.05s]
18:31:57  9 of 10 START test unique_dim_product_category_product_category ................ [RUN]
18:31:57  9 of 10 PASS unique_dim_product_category_product_category ...................... [PASS in 0.04s]
18:31:57  10 of 10 START test unique_fct_orders_order_id ................................. [RUN]
18:31:57  10 of 10 PASS unique_fct_orders_order_id ....................................... [PASS in 0.04s]
18:31:57
18:31:57  Finished running 10 data tests in 0 hours 0 minutes and 0.99 seconds (0.99s).
18:31:57
18:31:57  Completed successfully
18:31:57
18:31:57  Done. PASS=10 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=10
```

e. Buat dokumentasi dbt

```
(.venv) PS C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_miniproject> dbt do
cs generate --project-dir dbt_project --profiles-dir dbt_project
18:32:25  Running with dbt=1.12.6
18:32:26  Registered adapter: duckdb=1.11.0
18:32:28  Found 5 models, 10 data tests, 500 macros
18:32:28
18:32:28  Concurrency: 1 threads (target='dev')
18:32:28
18:32:29  Building catalog
18:32:29  Catalog written to C:\Users\vika\OneDrive\文件\SEMESTER 5\Data Processing Workflow\MiniProject\data_processing_m
iniproject\dbt_project\target\catalog.json
```

Kemudian jalankan:



Kesimpulan

Mini project *End-to-End Event-to-Warehouse Pipeline: Analisis Transaksi E-Commerce* bertujuan membangun alur pengolahan data transaksi dari proses ingestion hingga penyimpanan dan analisis data di data warehouse. Proyek ini mengintegrasikan Python, Pandas, Polars, Apache Kafka, Pydantic, DuckDB, dan dbt untuk mendukung proses pengolahan data yang terstruktur.

Pada tahap ingestion, data transaksi dibaca secara bertahap melalui simulasi API lokal menggunakan pagination, kemudian disimpan dalam format Parquet dengan mekanisme watermark untuk mendukung pemrosesan incremental. Selanjutnya, Pydantic digunakan untuk memvalidasi data berdasarkan kontrak yang telah ditentukan, sehingga data yang valid dapat dipisahkan dari data yang ditolak beserta alasan penolakannya.

Pada tahap streaming, Apache Kafka digunakan untuk mengirim dan menerima event transaksi secara real-time. Pengujian duplikasi menunjukkan pentingnya mekanisme idempotensi agar transaksi dengan order_id yang sama tidak dihitung berulang kali. Data yang lolos validasi kemudian diproses melalui transformasi, termasuk perhitungan gross_amount dan net_amount, serta evaluasi performa menggunakan Pandas dan Polars.

Pada tahap akhir, DuckDB digunakan sebagai data warehouse lokal untuk menyimpan data dalam bentuk staging, tabel fakta, tabel dimensi, dan agregasi penjualan harian. dbt mendukung

pengelolaan model transformasi SQL, pengujian kualitas data, serta pembuatan dokumentasi model.

Secara keseluruhan, proyek ini menunjukkan bagaimana berbagai komponen data engineering dapat diintegrasikan untuk membangun pipeline pengolahan transaksi yang modular dan dapat diuji. Namun, keberhasilan setiap tahap tetap perlu dibuktikan melalui hasil eksekusi, pengujian data, benchmark, dan tes dbt. Pengembangan selanjutnya dapat difokuskan pada penerapan idempotensi yang lebih kuat, otomatisasi pipeline secara menyeluruh, serta optimasi performa untuk menangani data dalam skala yang lebih besar.